

Instruction Set - Multiplication via Repeated Addition

Accumulator-Based CPU Model

What is Being Done?

The program multiplies:

$$AC := AC \times N$$

- **AC** initially contains the multiplicand (say Y).
- **N** is stored in memory location **mult**.
- Multiplication is done through repeated addition: $AC + AC + \dots$ (N times).

Program Logic

- Add **AC** to a product register **N** times.
- The loop:
 - Adds current **AC** to **prod**.
 - Decrements **N** (from **mult**) by 1.
 - Uses **BZ** (Branch if Zero) to exit when **N** becomes 0.
 - Otherwise, loops back to repeat.

Key Memory Locations

Label	Purpose
one	Constant value 1
mult	Multiplier N
ac	Initial value of AC (multiplicand Y)
prod	Final product/result

Assembly Program Breakdown

Line	Label	Instruction	Explanation
0	one	00...01	Memory location holding constant 1
1	mult	N	Multiplier N stored here
2	ac	00...00	Holds the multiplicand Y
3	prod	00...00	Holds the product (initially 0)
4		ST ac	Store AC (Y) into ac
5	Loop	LD mult	Load N into AC
6		BZ exit	Exit loop if $N == 0$
7		LD one	Load 1
8		MOV DR, AC	Move 1 into DR
9		LD mult	Reload N
10		SUB	Decrement N by 1
11		ST mult	Store updated N
12		LD ac	Load multiplicand (Y)
13		MOV DR, AC	Move Y into DR
14		LD prod	Load current product
15		ADD	Add Y to product
16		ST prod	Store updated product
17		BRA Loop	Jump back to loop
18	exit		End of program

Limitations of Accumulator-Based CPUs

1. **Single Accumulator:** Intermediate results must be moved to and from memory often.
2. **Limited Registers:**
 - Frequent memory access for values like 1, N , Y , and product.
3. **Slower Execution:**
 - Memory operations increase execution time.
 - If data could remain in CPU registers:
 - Program would run faster.
 - Fewer instructions needed.

Summary

- The program multiplies two values using repeated addition with an accumulator.
- It illustrates limitations of CPUs with few registers and no general-purpose registers.
- Efficiency can be greatly improved by using register-based architectures.